

Centro Universitario de Ciencias Exactas e Ingenierías

Interacción Humano Computadora



Hands-On 4: Actividad Integradora

Equipo:

Gándara Cornejo Ismael 220540311

Hernández Santos Karen Cecilia 219770168

Valentín Gallardo José Eduardo 218452685

Profesor: Jose Antonio Aviña Méndez

Carrera: Ingeniería en Computación

Fecha: 24 de mayo del 2026

Índice

Introducción	3
Transformaciones.....	4
1. Traslación	4
Fundamento Matemático	4
Segmento del Código Fuente Relacionado.....	4
2. Rotación	5
Fundamento Matemático	5
Segmento del Código Fuente Relacionado.....	5
3. Rebote.....	6
Fundamento Matemático	6
Segmento del Código Fuente Relacionado.....	6
4. Movimiento Senoidal	7
Fundamento Matemático	7
Segmento del Código Fuente Relacionado.....	7
5. Trayectoria Ondulada	8
Fundamento Matemático	8
Segmento del Código Fuente Relacionado.....	8
6. Órbita Circular	9
Fundamento Matemático	9
Segmento del Código Fuente Relacionado.....	9
Conclusiones	10
Referencias Bibliográficas.....	11

Introducción

En el ámbito de la computación gráfica, el desarrollo de videojuegos y la simulación interactiva en tiempo real, la representación visual de la realidad física depende fundamentalmente de la correcta traducción de modelos matemáticos abstractos a instrucciones de código ejecutable. La pantalla es una matriz bidimensional de píxeles; por lo tanto, la ilusión de profundidad, el movimiento fluido de los cuerpos y los cambios de orientación en un espacio tridimensional requieren el uso del álgebra lineal, el cálculo vectorial y las funciones trigonométricas.

Históricamente, los programadores debían interactuar directamente con las API de bajo nivel de las tarjetas gráficas (como OpenGL o Vulkan) para manipular los vértices de los objetos mediante matrices homogéneas. En la actualidad, bibliotecas modernas como *raylib* actúan como capas de abstracción que facilitan el renderizado tridimensional y el manejo de cámaras virtuales. Sin embargo, la lógica cinemática del comportamiento de los objetos dentro del bucle principal de la aplicación sigue siendo responsabilidad del desarrollador, quien debe diseñar algoritmos capaces de simular el espacio euclídeo de forma precisa y eficiente.

El presente reporte técnico aborda los fundamentos matemáticos y la implementación práctica en C++ de seis transformaciones utilizadas en entornos 3D: traslación lineal, rotación de matrices a través del stack de la GPU (r1gl), rebote por colisión elástica en fronteras delimitadas, movimiento armónico senoidal, trayectorias paramétricas compuestas y órbitas circulares uniformes, estas se muestran en la imagen 1. A través de este análisis, se demostrará cómo se vinculan las ecuaciones ideales de la física clásica con las variables discretas del código, validando el uso del tiempo del sistema como el eje para una simulación estable e independiente de la tasa de refresco por hardware.

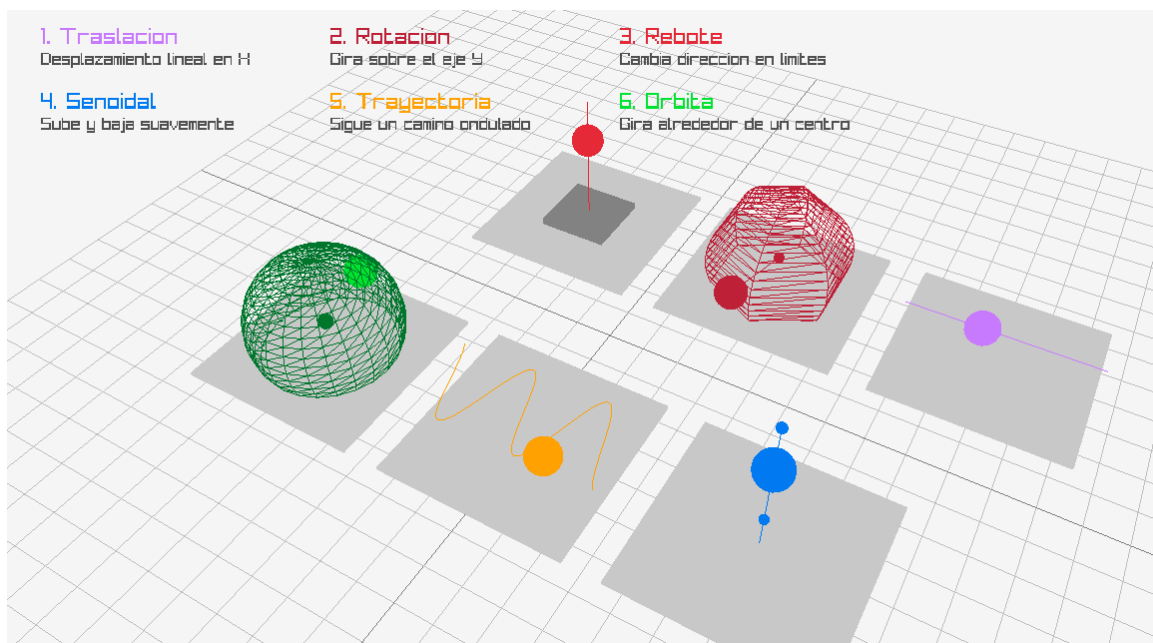


Imagen 1. Forma gráfica de las transformaciones matemáticas.

Transformaciones

1. Traslación

La traslación es una transformación afín que desplaza de forma lineal todos los puntos de un cuerpo rígido en una dirección y distancia uniformes en el espacio.

Fundamento Matemático

Dado un punto en coordenadas tridimensionales $P = (x, y, z)$, su posición trasladada $P' = (x', y', z')$ mediante un desplazamiento infinitesimal en el eje X provocado por una velocidad constante a lo largo de un intervalo de tiempo discreto se define formalmente como:

$$x_{k+1} = x_k + v_x$$

$$y_{k+1} = y_k$$

$$z_{k+1} = z_k$$

En coordenadas homogéneas, esta operación lineal se representa a través de la multiplicación del vector de posición original por la matriz de traslación global T:

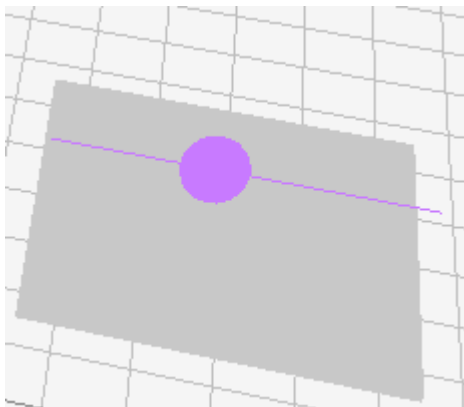
$$\begin{bmatrix} x' \\ y' \\ z' \\ 1 \end{bmatrix} = \begin{bmatrix} 1 & 0 & 0 & \Delta x \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} x \\ y \\ z \\ 1 \end{bmatrix}$$

Segmento del Código Fuente Relacionado

Variables: trasX (float).

```
// Actualización del diferencial de desplazamiento
trasX += 0.03f;
if (trasX > 2.5f) trasX = -2.5f;

// Renderizado en el espacio local de la Zona 1 (X global = -9.0f)
DrawSphere({ -9.0f + trasX, 1.0f, 4.0f }, 0.45f, PURPLE);
```



2. Rotación

La rotación modifica la orientación de un objeto geométrico respecto a un eje coordenado fijo, mapeando sus puntos en trayectorias circulares concéntricas.

Fundamento Matemático

La rotación en el espacio 3D implementada ocurre alrededor del eje vertical Y (rotación sobre el plano horizontal XZ). La matriz de rotación estándar $R_y(\theta)$ para un ángulo síncrono θ se define algebraicamente como:

$$R_y(\theta) = \begin{bmatrix} \cos(\theta) & 0 & \sin(\theta) & 0 \\ 0 & 1 & 0 & 0 \\ -\sin(\theta) & 0 & \cos(\theta) & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

Cuando se trabaja con transformaciones compuestas (rotar un objeto alrededor de un pivote desplazado), se aplica el producto de matrices en el orden inverso del pipeline gráfico (Local-to-World): $M = T_{\text{centro}} * R_y(\theta)$. Esto altera las posiciones locales del objeto transformándolo en:

$$x' = x \cos(\theta) + z \sin(\theta)$$

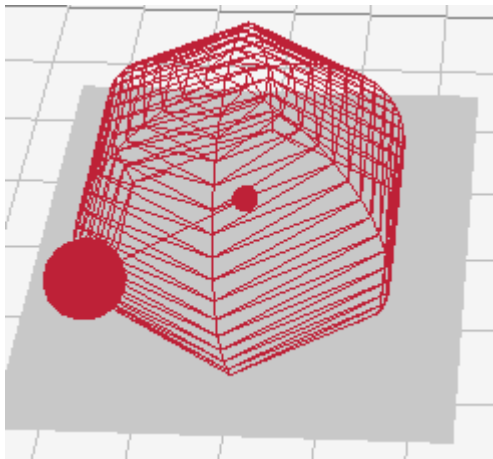
$$z' = -x \sin(\theta) + z \cos(\theta)$$

Segmento del Código Fuente Relacionado

Variables: ángulo (float).

```
// Incremento lineal discreto del ángulo de giro
ángulo += 1.0f;

// Manipulación del stack de matrices de la GPU mediante rlg1
rlPushMatrix();
    rlTranslatef(-3.0f, 1.0f, 4.0f); // Matriz de Traslación al centro de órbita
    rlRotatef(ángulo, 0.0f, 1.0f, 0.0f); // Matriz de Rotación pura sobre el eje Y
    DrawSphere({ 2.0f, 0.0f, 0.0f }, 0.45f, MAROON); // Desplazamiento del satélite
rlPopMatrix();
```



3. Rebote

El rebote modela una colisión cinemática elástica ideal contra una frontera o plano invisible, invirtiendo de manera instantánea el sentido de la componente ortogonal de la velocidad.

Fundamento Matemático

Cuando una partícula con un vector velocidad instantánea v choca contra un plano cuya frontera superior está acotada por y_{max} e inferior por y_{min} y su vector normal unitario es $n = (0, 1, 0)$, el principio de conservación de la energía mecánica (colisión elástica con coeficiente de restitución $e = 1$) dicta que la velocidad final v' se invierte:

$$\text{Si } y_k > y_{max} \text{ o } y_k < y_{min} \implies v' = v - 2(v \cdot n)n$$

En una sola dimensión lineal (eje Y), el operador vectorial colapsa a una multiplicación escalar por inversión de signo: $v_y' = -v_y$.

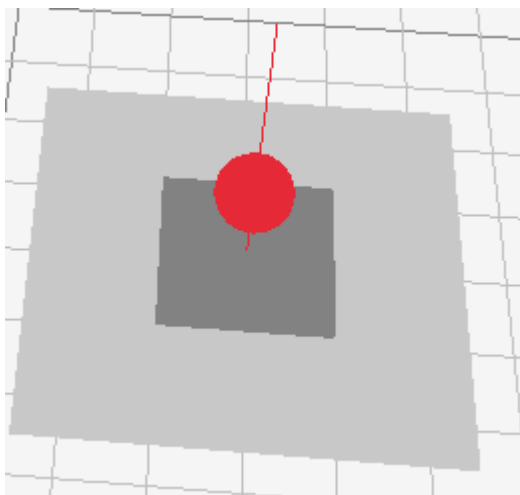
Segmento del Código Fuente Relacionado

Variables: reboteY (float), velocidadY (float).

```
// Integración de la posición vertical
reboteY += velocidadY;

// Condicional de frontera (Ecuación diferencial por tramos)
if (reboteY > 4.0f || reboteY < 1.0f) velocidadY *= -1.0f;

// Dibujo del elemento dinámico sujeto a límites
DrawSphere({ 3.0f, reboteY, 4.0f }, 0.45f, RED);
```



4. Movimiento Senoidal

Es un movimiento oscilatorio armónico unidimensional cuya posición varía de forma periódica en el tiempo siguiendo una función trigonométrica base.

Fundamento Matemático

La posición instantánea vertical $y(t)$ se describe mediante una ecuación matemática que vincula la amplitud física del fenómeno y su frecuencia angular en radianes, partiendo de un punto de equilibrio estático:

$$y(t) = y_0 + A \cdot \sin(\omega \cdot t + \phi)$$

Donde:

$y_0 = 2.0$ es la altura media de balanceo (punto de equilibrio).

$A = 1.5$ es la amplitud máxima de elongación de la onda senoidal.

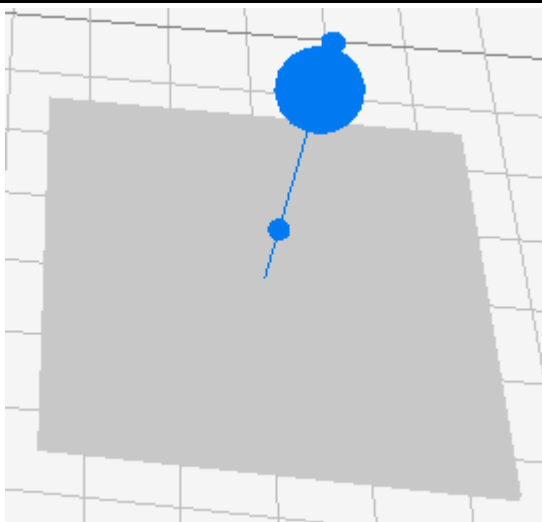
$\omega = 2.0$ es la frecuencia angular que regula la rapidez temporal de la oscilación.

t representa el tiempo continuo acumulado del sistema.

Segmento del Código Fuente Relacionado

Variables: t (float / tiempo acumulado de la aplicación), senoY (float).

```
float t = (float)GetTime();  
float senoY = 2.0f + sinf(t * 2.0f) * 1.5f;  
  
// Posicionamiento de la esfera en el eje Y dinámico  
DrawSphere({ -6.0f, senoY, -4.0f }, 0.45f, BLUE);
```



5. Trayectoria Ondulada

Combina un desplazamiento rectilíneo unidireccional y un movimiento armónico simultáneo en un eje perpendicular, trazando una curva espacial sinusoidal continua (onda estacionaria).

Fundamento Matemático

La posición bidimensional sobre el plano horizontal de la sub-escena viene parametrizada por el tiempo t a través del par ordenado $(x(t), z(t))$. La coordenada $x(t)$ está gobernada por un operador de módulo (truncamiento periódico) para acotar el trayecto dentro del espacio tridimensional de la celda, mientras que $z(t)$ modula la oscilación armónica:

$$x(t) = x_{inicio} + (v_x \cdot t) \pmod{L}$$

$$z(t) = A_z \cdot \sin(\omega_z \cdot t)$$

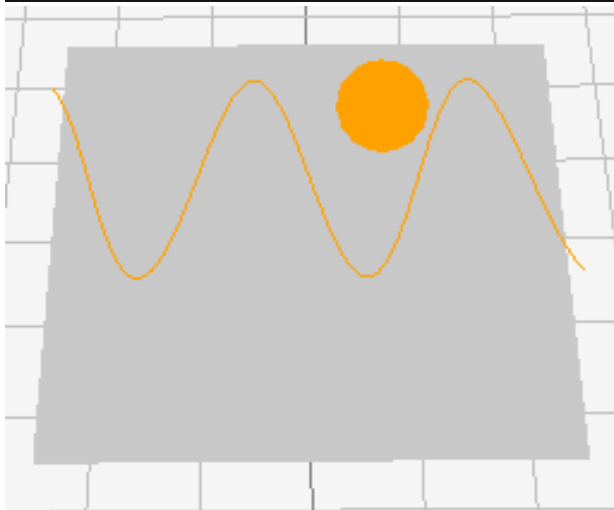
Donde $L = 5.0$ representa la longitud finita del carril geométrico recorrido y $\omega_z = 3.0$ la frecuencia de ondulación espacial.

Segmento del Código Fuente Relacionado

Variables: t (float), $trayectoriaX$ (float), $trayectoriaZ$ (float).

```
// Posición paramétrica temporal
float trayectoriaX = -2.5f + fmodf(t * 1.5f, 5.0f);
float trayectoriaZ = sinf(t * 3.0f) * 1.2f;

// Asignación de coordenadas espaciales al objeto gráfico
DrawSphere({ 0.0f + trayectoriaX, 1.0f, -4.0f + trayectoriaZ }, 0.45f, ORANGE);
```



6. Órbita Circular

Es una trayectoria bidimensional cerrada donde un punto se traslada de manera continua alrededor de un foco central, conservando un radio de separación equidistante.

Fundamento Matemático

El movimiento orbital circular uniforme se modela mediante la conversión cinemática directa de coordenadas polares a coordenadas rectangulares (cartesianas) sobre el plano horizontal XZ. Su formulación analítica es la siguiente:

$$x(t) = R \cdot \cos(\theta(t)) = R \cdot \cos(\omega \cdot t)$$

$$z(t) = R \cdot \sin(\theta(t)) = R \cdot \sin(\omega \cdot t)$$

Donde:

$R = 2.0$ es la distancia constante (radio de la circunferencia).

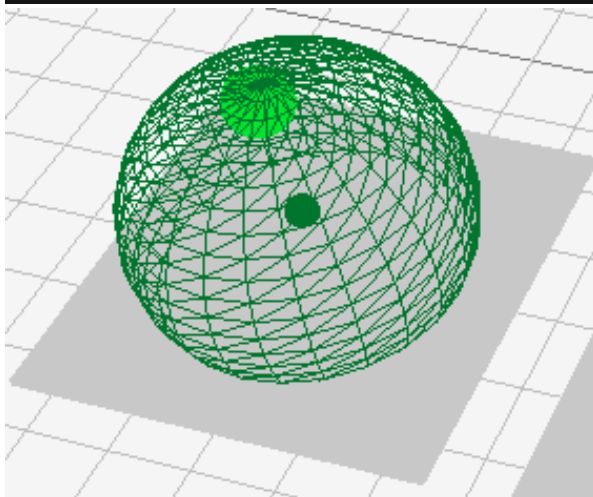
$\omega = 1.0$ rad/s es la velocidad angular del cuerpo respecto al centro debido a que el argumento del ángulo equivale directamente al tiempo de ejecución t .

Segmento del Código Fuente Relacionado

Variables: radio (float), orbitaX (float), orbitaZ (float), t (float).

```
float radio    = 2.0f;
float orbitaX  = cosf(t) * radio;
float orbitaZ  = sinf(t) * radio;

// Renderizado desplazado respecto al foco orbital central de la Zona 6 (X=6.0, Z=-4.0)
DrawSphere({ 6.0f + orbitaX, 1.0f, -4.0f + orbitaZ }, 0.45f, GREEN);
```



Conclusiones

Al realizar este proyecto práctico e integrador nos permitió tener diversas conclusiones sobre la relación que existe entre las ciencias matemáticas puras y la ingeniería de software gráfico en tiempo real:

Abstracción Gráfica y Simulación Científica: Se demuestra que cualquier comportamiento cinemático o alteración espacial en un entorno tridimensional por más orgánico o complejo que parezca visualmente es reducible a operaciones algebraicas elementales, funciones trigonométricas y transformaciones lineales. La pantalla no es más que un lienzo donde la profundidad es una ilusión matemática calculada mediante matrices de proyección; por ende, dominar la geometría analítica es el único camino viable para controlar el comportamiento de los objetos en un motor de videojuegos o simulador gráfico.

Optimización mediante el Stack de Matrices (GPU vs. CPU): La coexistencia de dos metodologías de transformación en el código (una basada en cálculos manuales explícitos sobre variables de posición como `trasX` o `orbitaX`, y otra basada en las funciones de `rlgl.h` como `rlRotatef` y `rlTranslatef`) pone de manifiesto la arquitectura moderna de renderizado. Mientras que calcular las posiciones en la CPU es útil para la lógica del juego (física, colisiones, IA), delegar las transformaciones de rotación complejas a la GPU mediante matrices fijas optimiza drásticamente el rendimiento del procesador central. Esto evita recalcular cada vértice de un modelo tridimensional en el ciclo principal de la aplicación.

Sincronización Temporal y Estabilidad: El uso de la función `GetTime()` para modular el movimiento senoidal, la trayectoria y la órbita introduce el concepto de estabilidad cinemática. Desvincular el movimiento de los fotogramas por segundo mediante variables temporales continuas es un estándar mandatorio. Esto garantiza que la velocidad y el comportamiento físico de los elementos se procesen de manera idéntica sin importar la capacidad de cómputo o los retrasos de hardware del dispositivo de ejecución.

El Rol de la Trigonometría en Entornos Virtuales: El análisis de los módulos de órbita, movimiento armónico y trayectorias onduladas hace ver que las funciones circulares son los motores fundamentales de la periodicidad, la oscilación y las fuerzas de sujeción. La conversión del espacio polar al espacio cartesiano euclidiano es el pilar de la mecánica celeste, la simulación de fluidos y la animación de personajes dentro de cualquier motor gráfico contemporáneo. En conclusión, el desarrollo de esta aplicación interactiva demuestra la teoría vista en clase, demostrando que la programación gráfica son matemáticas aplicadas en movimiento.

Referencias Bibliográficas

- Hughes, J. F., Van Dam, A., McGuire, M., Sklar, D. F., Foley, J. D., Feiner, S. K., & Akeley, K. (2014). *Computer Graphics: Principles and Practice* (3rd ed.). Boston, MA: Addison-Wesley Professional.
- Lengyel, E. (2016). *Mathematics for 3D Game Programming and Computer Science* (3rd ed.). Boston, MA: Cengage Learning.
- Stewart, J. (2012). *Cálculo de una variable: Trascendentes tempranas* (7th ed.). Ciudad de México, México: Cengage Learning.